Développement d'une DApp avec Truffle et Ganache

Filière : IA et BigData Niveau : 4è année

Réalisé par :

- Audes Mariana MOUSSAVOU MOUSSAVOU
- Stécy MOMBO AZZILE

Encadré par :

- Kebira AZBEG

Année universitaire 2025-2026

Sommaire

Partie 1 : Développement d'une Dapp Simple 5
1. Phase 1 : Installation des outils 5
a. Node.js & npm 5
b. Installation de Truffle 5
c. Installation de Ganache 6
2. Phase 2 : Initialisation du projet truffle 6
a. Initialisation de npm et Truffle 7
3. Phase 3 : Lancement de Ganache (blockchain locale) 8
4. Phase 5 : Préparation de la migration 9
a. Création des migrations 9
5. Phase 6 : Configuration de Truffle 10
6. Phase 7 : Déploiement du contrat 10
7. Phase 8 : Interaction via Truffle Console 11
8. Phase 9 : Tests unitaires 11
9. Phase 10 : Déploiement & test via Remix + MetaMask 12
10. Phase 11 : Création d'une interface Web minimale 13
Partie 2 : Mini projet : Développement d'une DApp de gestion des annonces de la classe 14
1. Création du fichier solidity 14
2. Config des migrations 15
3. Compilation 15
4. Déploiement 15
5. Test 16
6. Déploiement & test via Remix 17
7. **Création d'une interface Web minimale** 18
CONCLUSION 20

Table des figures

Figure 1: Vérification des versions de node et npm 5
Figure 2: Installation de truffle 6
Figure 3: Installation de ganache 6
Figure 4: Initialisation de npm 7
Figure 5: : Initialisation de truffle 7
Figure 6: Structure du projet 8
Figure 7: visualisation de la migration créée 9
Figure 8: Configuration de truffle 10
Figure 9: Déploiement du contrat 10
Figure 10: Interaction via Truffle Console 11
Figure 11: Tests unitaires 1 11
Figure 12: Tests unitaires 2 12
Figure 13:1.Déploiement & test via Remix + MetaMask 12
Figure 14: Création d'une interface Web minimale 1 13
Figure 15: Interface Web minimale 13
Figure 16 : 1. Création du fichier solidity 14
Figure 17 : Config des migrations 15
Figure 18:Compilation 15
Figure 19: Déploiement 16
Figure 20: Test 1 16
Figure 21: Test 2 17
Figure 22: Déploiement & test via Remix 18

INTRODUCTION

La blockchain est souvent présentée comme une révolution technologique, mais avant tout c'est une idée simple : un registre partagé, où chaque information inscrite est validée par le réseau et ne peut plus être modifiée. Pas de serveur central, pas d'autorité unique : la confiance vient du système lui-même. Cette approche change la manière dont on échange de la valeur, des données ou même des messages, et ouvre la porte à des applications décentralisées (**DApps**) qui fonctionnent sans intermédiaire.

Son importance est aujourd'hui visible dans des domaines variés : finance, logistique, santé, mais aussi dans des contextes plus proches de nous, comme l'éducation. Comprendre la blockchain, ce n'est pas seulement apprendre une nouvelle technologie, c'est aussi se préparer à des usages concrets où transparence et traçabilité deviennent essentielles.

Le mini-projet que nous avons réalisé s'inscrit dans cette logique. Après une première partie consacrée à un contrat simple (**SimpleStorage**), nous avons développé une application décentralisée permettant de gérer les annonces de classe. L'idée est de montrer, à travers un cas pratique, comment un smart contract peut servir à publier, consulter et sécuriser des informations utiles au quotidien. Ce projet est donc à la fois un exercice pédagogique et une démonstration de l'intérêt réel de la blockchain dans un cadre académique.

Partie 1 : Développement d'une Dapp Simple

Après avoir défini la blockchain et son intérêt, il est essentiel de passer à une première mise en pratique. Cette partie du travail est consacrée au développement d'une application décentralisée simple, qui servira de base pour comprendre les outils et les étapes nécessaires avant d'aborder un projet plus complet.

L'objectif est de créer un contrat intelligent minimaliste (**SimpleStorage**) permettant de stocker et de lire une valeur sur une blockchain locale. Ce choix pédagogique est pertinent car il permet de se familiariser avec les concepts fondamentaux sans complexité excessive : compilation, migration, interaction et tests.

Cette partie est structurée en plusieurs phases, chacune illustrée par des captures d'écran (figures) qui montrent concrètement les étapes réalisées.

1. Phase 1 : Installation des outils

La première étape consiste à installer l'environnement nécessaire au développement d'une DApp. Cela inclut **Node.js** et son gestionnaire de paquets **npm**, qui permettent d'exécuter du JavaScript côté serveur et de gérer les dépendances du projet.

a. Node.js & npm

```
C:\Users\22891>node --version
v20.18.2

C:\Users\22891>npm --version
11.12.1
```

Figure 1: Vérification des versions de node et npm

Ensuite, on installe **Truffle**, un framework spécialisé pour la compilation, le déploiement et le test des smart contracts. Enfin, **Ganache** est mis en place pour simuler une blockchain Ethereum locale avec des comptes préchargés en ETH fictifs. L'intérêt de cette phase est de disposer d'une base technique fiable et complète, sans dépendre d'un réseau public, afin de travailler en toute autonomie.

b. Installation de Truffle

```
C:\Users\22891>npm install -g truffle
```

```
C:\Users\22891>truffle --version
Truffle v5.11.5 (core: 5.11.5)
Ganache v7.9.1
Solidity v0.5.16 (solc-js)
Node v20.18.2
Web3.js v1.10.0
```

Figure 2: Installation de truffle

c. Installation de Ganache

```
C:\Users\22891>npm install -g ganache

added 336 packages in 19s
```

```
C:\Users\22891>ganache --version
ganache v7.9.2 (@ganache/cli: 0.10.2, @ganache/core: 0.10.2)
```

Figure 3: Installation de ganache

2. Phase 2 : Initialisation du projet truffle

Une fois les outils installés, il faut créer la structure du projet. On initialise d'abord npm avec npm init, puis Truffle avec truffle init. Cela génère une arborescence standard composée des dossiers **contracts/** (pour les smart contracts), **migrations/** (pour les scripts de déploiement), **test/** (pour les tests unitaires) et du fichier **truffle-config.js** (pour la configuration réseau). L'intérêt est d'avoir une organisation claire et reproductible, conforme aux bonnes pratiques du développement blockchain.

```
C:\Users\22891>mkdir C:\Users\22891\tp-blockchain

C:\Users\22891>cd C:\Users\22891\tp-blockchain
```

a. Initialisation de npm et Truffle

```
C:\Users\22891\tp-blockchain>npm init -y
Wrote to C:\Users\22891\tp-blockchain\package.json:

{
  "name": "tp-blockchain",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

Figure 4: Initialisation de npm

```
C:\Users\22891\tp-blockchain>truffle init

Starting init...
================

> Copying project files to C:\Users\22891\tp-blockchain

Init successful, sweet!

Try our scaffold commands to get started:
  $ truffle create contract YourContractName # scaffold a contract
  $ truffle create test YourTestName         # scaffold a test

http://trufflesuite.com/docs
```

Figure 5: : Initialisation de truffle

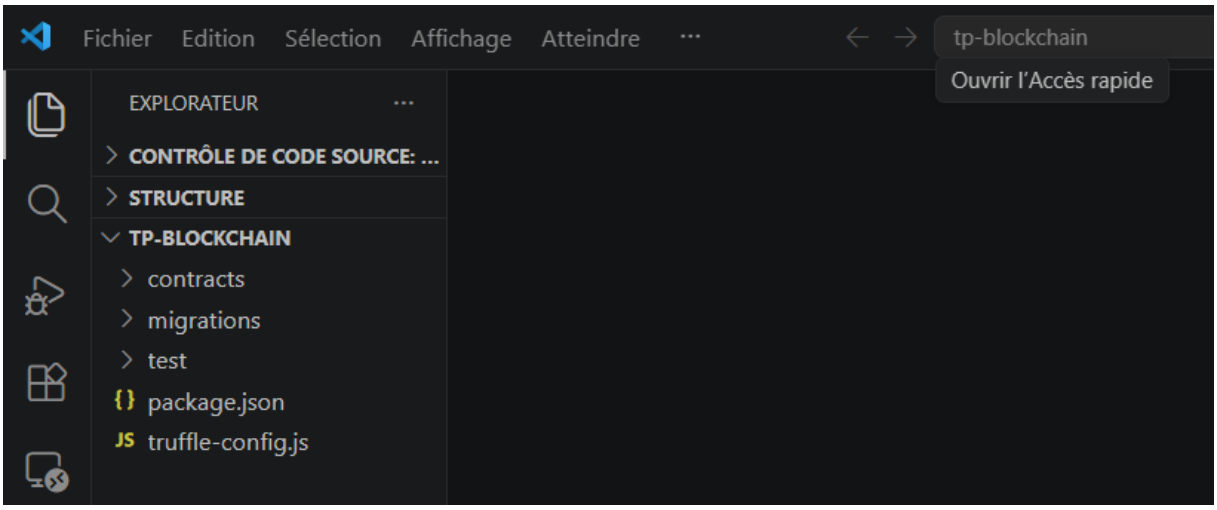


Figure 6: Structure du projet

3. Phase 3 : Lancement de Ganache (blockchain locale)

Ganache est ensuite lancé pour fournir une blockchain locale. Il crée automatiquement plusieurs comptes avec des soldes fictifs en Ether et mine instantanément chaque transaction. Cela permet de simuler un environnement Ethereum réaliste, mais contrôlé, où l'on peut tester librement sans frais ni délais. L'intérêt est de pouvoir expérimenter et déboguer rapidement, sans dépendre d'un testnet public comme Sepolia ou Goerli.

```
C:\Users\22891\tp-blockchain>ganache --verbose --db ./ganache-db --accounts 10 -- defaultBalanceEther 100 --port 8545
ganache v7.9.2 (@ganache/cli: 0.10.2, @ganache/core: 0.10.2)
Starting RPC server
Migrating database from version `null` to `0`…
Migration complete

Available Accounts
==================
(0) 0x00BE53325cA5501456F82588A166DB437BCD7903 (1000 ETH)
(1) 0xC6b49d4a08861457a2b509d980081bC5B473ca2D (1000 ETH)
(2) 0x2F2040aAEE7c7A1B94706A528df4b3CD641498b3 (1000 ETH)
(3) 0x054a6552992CEfE6614c6F1e10063f3fd768f8eF (1000 ETH)
(4) 0xe001CbD2554789C2e91e20D36AEF6Ee8E761E948 (1000 ETH)
(5) 0xdc23f1aE575E1e88A5B303168217E0cB8f21881D (1000 ETH)
(6) 0x62af36FF80885Ae1C7C3ff013dcA8d3BAdd129c2 (1000 ETH)
(7) 0xf77219996639d8db19E852feFfF1976Ac87e35F3 (1000 ETH)
(8) 0x80E98C20d67693127e125cC02c60373b334525eD (1000 ETH)
(9) 0xebb5894F78Da747a591B5607C0C0154147aa642a (1000 ETH)

Private Keys
==================
(0) 0x80354f5cc281c2912365c3b8db57df40ea8a902a6fcfda0704d529cc5bd4f5ee
(1) 0xff46a6e2a62590154def337dcc9cae5ef87f59b4c92d783d46ae93163623879e
(2) 0xd11fe0f94d6a450f855d4e4a97c023533875622c519cca7b69f7e0450315bd86
(3) 0xfe392f4f9023f58679b5c923e73326992ed5519ee7c9d1296696f138db0de999
(4) 0x7c93940e3c77e7ead26b3656a5966dc2b949bb36413bdaf827798a8fe1ef6e6e
(5) 0xf9104f0f9971ed41d9e5ddc5bd0ee09061d542014de1d12792092dc119438b87
(6) 0xde3a6c401814f6df4ca9d4b070139d072a78d0af92df6ddb501a0cf5dff68f22
(7) 0xcb4572942f58f4c0806bb9972f540fd3b76fd2e8feb938375938fbfcc83a8692
(8) 0xb73e3e457face196349e3c5802ccf794ade192f10536e9f712749a162e727299
(9) 0x314a02cea4e5ca6aa96a0339aa50314bca2fd4e6ffd074e3e01dddbaf0420fc9
```

```
HD Wallet
==================
Mnemonic:      nuclear accident address mention violin shop kit loop fuel garage parrot vital
Base HD Path:  m/44'/60'/0'/0/{account_index}

Default Gas Price
==================
2000000000

BlockGas Limit
==================
30000000

Call Gas Limit
==================
50000000

Chain
==================
Hardfork: shanghai
Id:       1337

RPC Listening on 127.0.0.1:8545
>
```

4. Phase 5 : Préparation de la migration

Avant de déployer un contrat, il faut préparer les scripts de migration. Ces fichiers JavaScript décrivent comment Truffle doit installer les contrats sur la blockchain. On configure également le fichier **truffle-config.js** pour relier Truffle à Ganache (adresse `127.0.0.1:8545`). L'intérêt est d'automatiser le déploiement et de garantir que chaque contrat est installé correctement, avec une traçabilité des versions.

a. Création des migrations

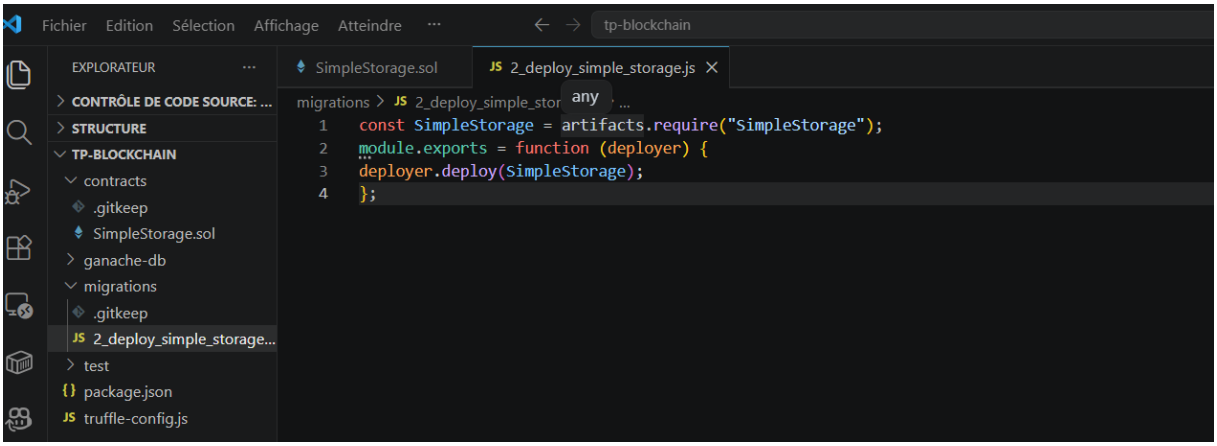


Figure 7: visualisation de la migration créée

5. Phase 6 : Configuration de Truffle

Le contrat Solidity est compilé avec truffle compile, ce qui génère un artefact JSON contenant son ABI et son bytecode. Ensuite, truffle migrate déploie le contrat sur Ganache. Cette étape confirme que le code est valide et que le contrat est bien enregistré sur la blockchain locale. L'intérêt est de rendre le contrat opérationnel et accessible pour les interactions futures.

```
JS truffle-config.js
118      }
119    },
120    development: {
121      host: "127.0.0.1",
122      port: 8545,
123      network_id: "*"
124    },
```

Figure 8: Configuration de truffle

6. Phase 7 : Déploiement du contrat

Ensuite, truffle migrate déploie le contrat sur Ganache. Cette étape confirme que le code est valide et que le contrat est bien enregistré sur la blockchain locale. L'intérêt est de rendre le contrat opérationnel et accessible pour les interactions futures.

```
PS C:\Users\22891\tp-blockchain> truffle migrate --network development
Compiling your contracts...
===========================
> Compiling .\contracts\SimpleStorage.sol
> Artifacts written to C:\Users\22891\tp-blockchain\build\contracts
> Compiled successfully using:
   - solc: 0.8.21+commit.d9974bed.Emscripten.clang


Starting migrations...
======================
> Network name:    'development'
> Network id:      1776400928644
> Block gas limit: 30000000 (0x1c9c380)


2_deploy_simple_storage.js
==========================

   Deploying 'SimpleStorage'
   -------------------------
   > transaction hash:    0x2a9f0ad1d3b27b1b9f6c66fd2352155c99148d62a615567f8a43e6283d552de9
   > Blocks: 0            Seconds: 0
   > contract address:    0xe78A0F7E598Cc8b0Bb87894B0F60dD2a88d6a8Ab
   > block number:        1
   > block timestamp:     1776401134
   > account:             0x90F8bf6A479f320ead074411a4B0e7944Ea8c9C1
   > balance:             999.999604993375
   > gas used:            117039 (0x1c92f)
   > gas price:           3.375 gwei
   > value sent:          0 ETH
   > total cost:          0.000395006625 ETH

   > Saving artifacts
   -------------------------------------
   > Total cost:      0.000395006625 ETH

Summary
=======
> Total deployments:   1
> Final cost:          0.000395006625 ETH
```

Figure 9: Déploiement du contrat

7. Phase 8 : Interaction via Truffle Console

Une fois déployé, le contrat peut être manipulé via la **Truffle console**. On peut appeler ses fonctions, par exemple `set()` pour écrire une valeur et `get()` pour la lire. Cette interaction directe en JavaScript permet de tester le comportement du contrat sans interface graphique. L'intérêt est de vérifier rapidement la logique et de comprendre comment les transactions modifient l'état du contrat.

```
truffle(development)> await inst.set(123)
{
  tx: '0x27ce6ec0d0234b4b73abc54cdf0433c49e8ad0db0c046781f24e52c8fd35df57',
  receipt: {
    transactionHash: '0x27ce6ec0d0234b4b73abc54cdf0433c49e8ad0db0c046781f24e52c8fd35df57',
    transactionIndex: 0,
    blockNumber: 2,
    blockHash: '0x8064bd03923269e9c35d3dce6c5d065721cbb0803608df9a3680543bf4559777',
    from: '0x90f8bf6a479f320ead074411a4b0e7944ea8c9c1',
    to: '0xe78a0f7e598cc8b0bb87894b0f60dd2a88d6a8ab',
    cumulativeGasUsed: 43696,
    gasUsed: 43696,
    contractAddress: null,
    logs: [],
    logsBloom: '0x00000000000
```

```
PS C:\Users\22891\tp-blockchain> truffle test --network development
Using network 'development'.


Compiling your contracts...
===========================
> Compiling .\contracts\SimpleStorage.sol
> Artifacts written to C:\Users\22891\AppData\Local\Temp\test--19060-2bwuYRLmtDVm
> Compiled successfully using:
   - solc: 0.8.21+commit.d9974bed.Emscripten.clang


  Contract: SimpleStorage
    ✓ stocke et renvoie la bonne valeur (46ms)


  1 passing (68ms)
```

Figure 12: Tests unitaires 2

9. Phase 10 : Déploiement & test via Remix + MetaMask

Au-delà de Truffle, le contrat est testé dans **Remix IDE**, un environnement en ligne qui permet de compiler et d'interagir avec des smart contracts. Les transactions sont signées via **MetaMask**, un portefeuille numérique qui simule l'expérience utilisateur réelle. L'intérêt est de valider le contrat dans un contexte plus proche de la production, avec une interface graphique et une gestion des comptes.

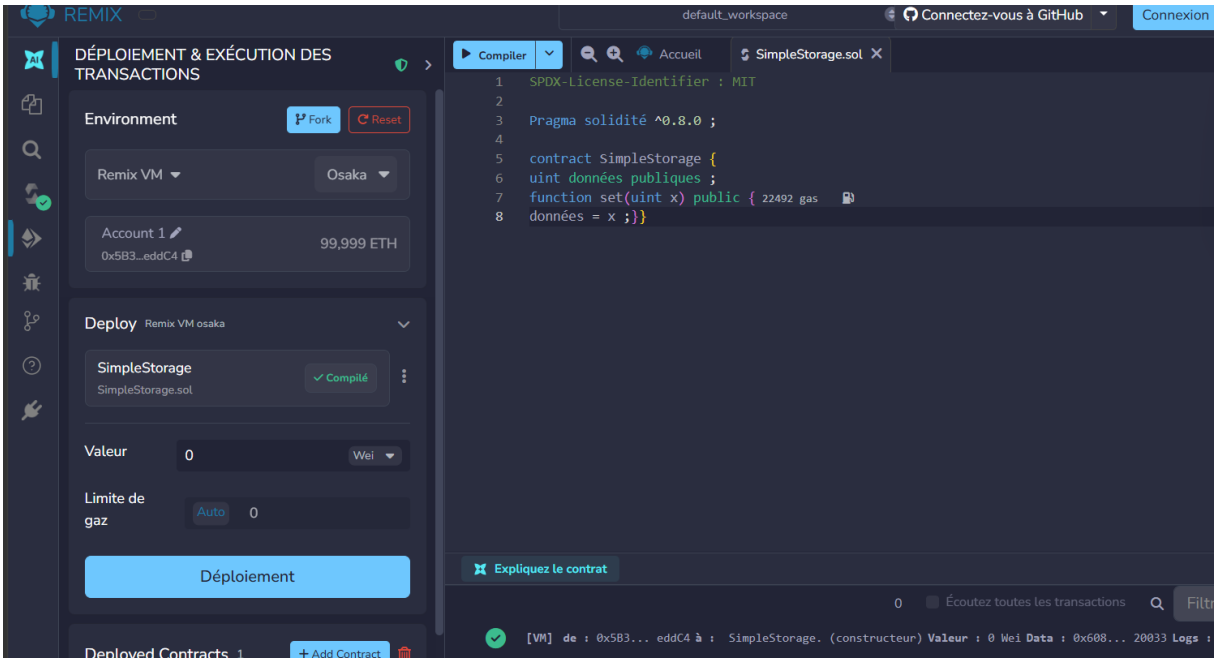


Figure 13:Déploiement & test via Remix + MetaMask

10. Phase 11 : Création d'une interface Web minimale

Enfin, une page HTML utilisant **Web3.js** est développée pour interagir avec le contrat via MetaMask. L'utilisateur peut saisir une valeur, l'envoyer au contrat, puis la lire. Cette interface simple illustre comment une DApp peut être utilisée par un utilisateur final. L'intérêt est de montrer la concrétisation du projet : un contrat déployé sur la blockchain, accessible via une interface web intuitive.

Figure 14: Création d'une interface Web minimale 1

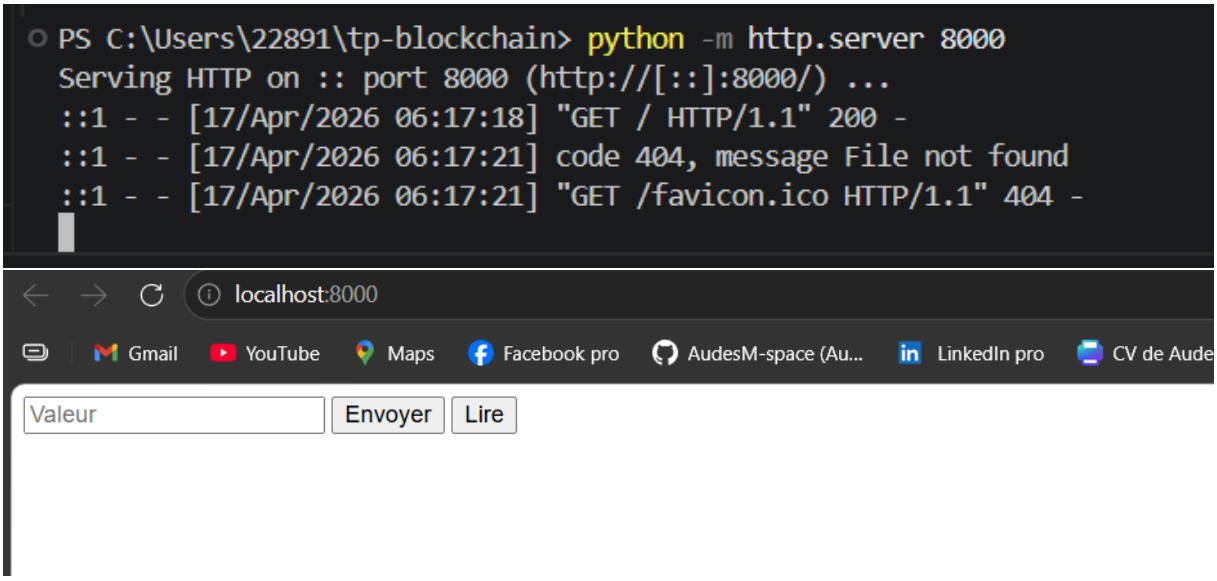


Figure 15: Interface Web minimale

Partie 2 : Mini projet : Développement d'une DApp de gestion des annonces de la classe

Après avoir expérimenté avec un contrat simple dans la première partie, il est temps de passer à un projet plus concret et représentatif des usages possibles de la blockchain. Cette deuxième partie est consacrée au développement d'une **DApp de gestion des annonces de la classe**. L'objectif est de concevoir un smart contract capable de publier, consulter et compter des annonces, puis de l'intégrer dans une interface web minimale. Ce mini-projet illustre comment une logique décentralisée peut répondre à un besoin réel : partager des informations de manière transparente et sécurisée entre les membres d'une classe. Il permet ainsi de mettre en pratique les compétences acquises, tout en démontrant l'intérêt de la blockchain dans un contexte académique.

1. Création du fichier solidity

Dans cette étape, nous écrivons le smart contract Annonce (ou ClassAnnouncements) en langage Solidity. Ce contrat définit la structure d'une annonce (identifiant, message, auteur, date) et les fonctions pour en ajouter une, en consulter une par identifiant et obtenir le nombre total d'annonces. L'intérêt est de traduire un besoin concret – la gestion des annonces de classe – en logique blockchain, ce qui permet de comprendre comment un cas d'usage réel se formalise dans un contrat intelligent.

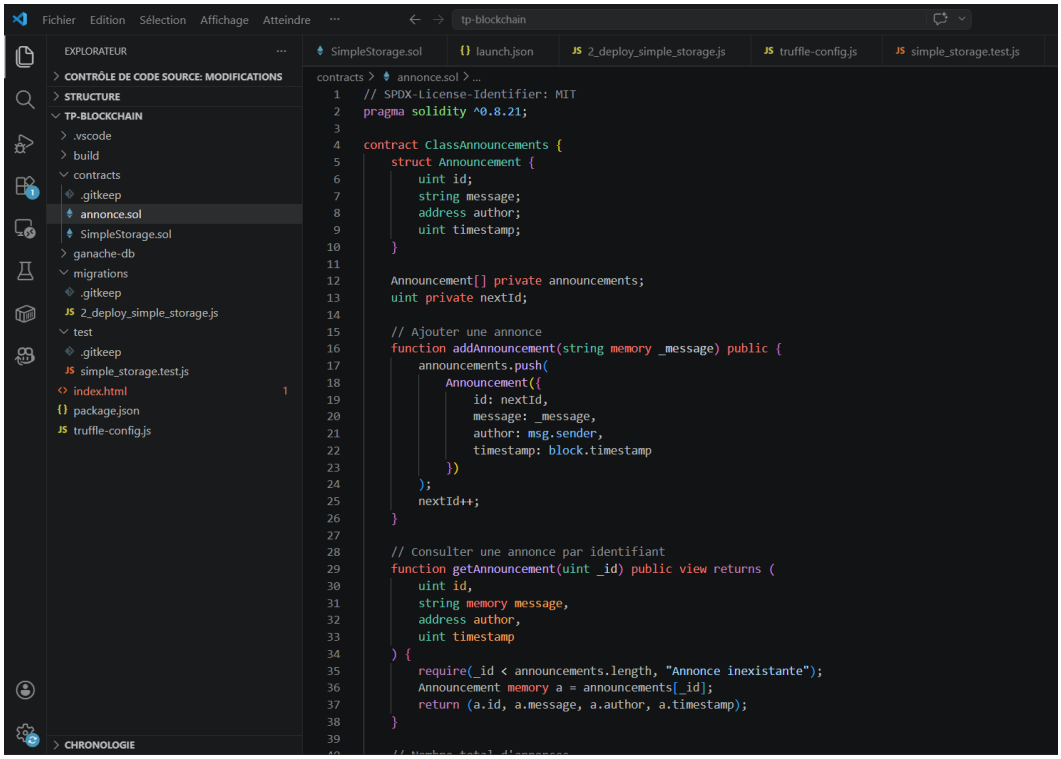


Figure 16 : 1. Création du fichier solidity

2. Config des migrations

Une fois le contrat écrit, il faut préparer son déploiement en créant un fichier de migration, par exemple 3_deploy_annonce.js. Ce script indique à Truffle comment installer le contrat sur la blockchain locale. On ajuste aussi le fichier truffle-config.js pour s'assurer que la connexion avec Ganache est correcte. L'intérêt est d'automatiser le déploiement et de garantir que le contrat est installé de manière reproductible.

Figure 17 : Config des migrations

```
SimpleStorage.sol   launch.json   2_deploy_simple_storage.js   truffle-config.js
migrations > 2_deploy_simple_storage.js > <unknown> > exports
1
2   const Annonce = artifacts.require("annonce");
3
4   module.exports = function (deployer) {
5
6   deployer.deploy(Annonce);
7   };
```

3. Compilation

Le contrat est ensuite compilé avec la commande truffle compile. Cette opération génère un fichier JSON contenant l'ABI et le bytecode du contrat, indispensables pour interagir avec lui via Web3.js ou Truffle. L'intérêt est de transformer le code Solidity en un format compréhensible par la machine virtuelle Ethereum (EVM).

```
PS C:\Users\22891\tp-blockchain> truffle compile

Compiling your contracts...
===========================
> Compiling .\contracts\SimpleStorage.sol
> Compiling .\contracts\annonce.sol
> Artifacts written to C:\Users\22891\tp-blockchain\build\contracts
> Compiled successfully using:
   - solc: 0.8.21+commit.d9974bed.Emscripten.clang
```

Figure 18:Compilation

4. Déploiement

Après compilation, on déploie le contrat sur Ganache avec truffle migrate. Cette étape enregistre le contrat sur la blockchain locale et lui attribue une adresse unique. L'intérêt est de rendre le contrat accessible pour les tests et les interactions, comme si nous étions sur un vrai réseau Ethereum.

```
PS C:\Users\22891\tp-blockchain> truffle migrate --network development

Compiling your contracts...
===========================
> Compiling .\contracts\SimpleStorage.sol
> Compiling .\contracts\annonce.sol
> Compiling .\contracts\annonce.sol
> Artifacts written to C:\Users\22891\tp-blockchain\build\contracts
> Compiled successfully using:
   - solc: 0.8.21+commit.d9974bed.Emscripten.clang


Starting migrations...
======================
> Network name:    'development'
> Network id:      1776404254348
> Block gas limit: 30000000 (0x1c9c380)


2_deploy_simple_storage.js
==========================

   Deploying 'Annonce'
   -------------------
   > transaction hash:    0xbcad95db1bb0b0ba6dbbbe18752731590d9692b11b611d5e8fcb48320760a710
   > Blocks: 0            Seconds: 0
   > contract address:    0xe78A0F7E598Cc8b0Bb87894B0F60dD2a88d6a8Ab
   > block number:        1
   > block timestamp:     1776404277
   > account:             0x90F8bf6A479f320ead074411a4B0e7944Ea8c9C1
   > balance:             999.997890112
   > gas used:            625152 (0x98a00)
   > gas price:           3.375 gwei
   > value sent:          0 ETH
   > total cost:          0.002109888 ETH

   > Saving artifacts
   -------------------------------------
   > Total cost:         0.002109888 ETH

Summary
=======
```

Figure 19: Déploiement

5. Test

Pour vérifier le bon fonctionnement, des tests unitaires sont écrits en JavaScript. Ils permettent de s'assurer qu'une annonce ajoutée est bien stockée, que la lecture par identifiant renvoie les bonnes informations et que le compteur d'annonces est exact. L'intérêt est de garantir la fiabilité du contrat et d'éviter les erreurs logiques avant un déploiement plus large.

```
Fichier  Edition  Sélection  Affichage  Atteindre  ...   tp-blockchain

EXPLORATEUR                 SimpleStorage.sol | launch.json | 2_deploy_simple_storage.js | classAnnouncements.test.js
CONTRÔLE DE CODE SOURCE: MODIFICATIONS
STRUCTURE                   test > classAnnouncements.test.js > ...
TP-BLOCKCHAIN
  .vscode                   1   const Annonce: any
  build                     2
  contracts                 3   const Annonce = artifacts.require("annonce");
    .gitkeep                4
    annonce.sol             5   contract("Annonce", accounts => {
    SimpleStorage.sol       6     it("ajoute et lit une annonce", async () => {
  ganache-db                7       const inst = await Annonce.deployed();
  migrations                8       await inst.addAnnouncement("Première annonce", { from: accounts[0] });
    .gitkeep                9       const total = await inst.totalAnnouncements();
    2_deploy_simple_storage.js 10   assert.equal(total.toNumber(), 1, "Il doit y avoir 1 annonce");
  test                      11
    .gitkeep                12      const ann = await inst.getAnnouncement(0);
                            13      assert.equal(ann.message, "Première annonce", "Le message doit correspondre");
                            14      assert.equal(ann.author, accounts[0], "L'auteur doit être accounts[0]");
                            15    });
                            16  });
```

Figure 20: Test 1

```
PS C:\Users\22891\tp-blockchain> truffle test --network development
> Compiling .\contracts\annonce.sol
> Compiling .\contracts\annonce.sol
> Artifacts written to C:\Users\22891\AppData\Local\Temp\test--6164-VmJjhm9JO50c
> Compiled successfully using:
   - solc: 0.8.21+commit.d9974bed.Emscripten.clang


  Contract: Annonce
    ✓ ajoute et lit une annonce (97ms)
```

Figure 21: Test 2

6. Déploiement & test via Remix

Au-delà de Truffle, le contrat est testé dans Remix IDE, un environnement en ligne qui permet de compiler et d'interagir avec des smart contracts. Les transactions sont signées via MetaMask, ce qui simule l'expérience utilisateur réelle. L'intérêt est de valider le contrat dans un contexte proche de la production, avec une interface graphique et une gestion des comptes.

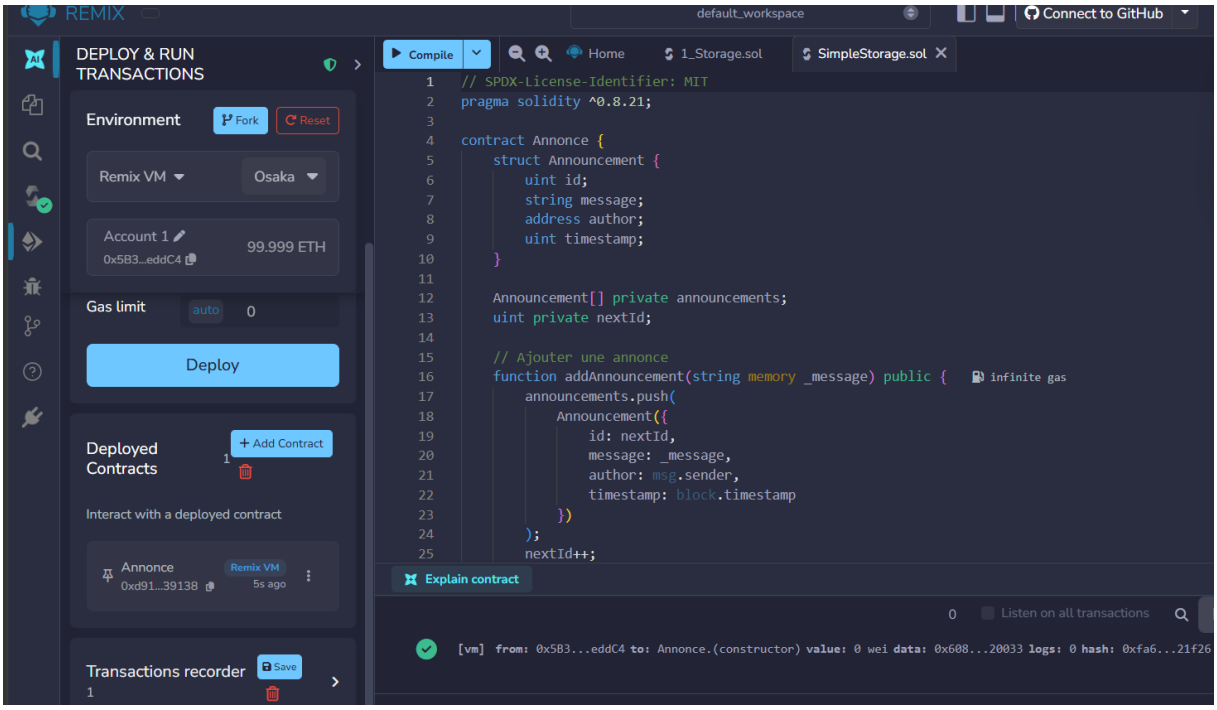
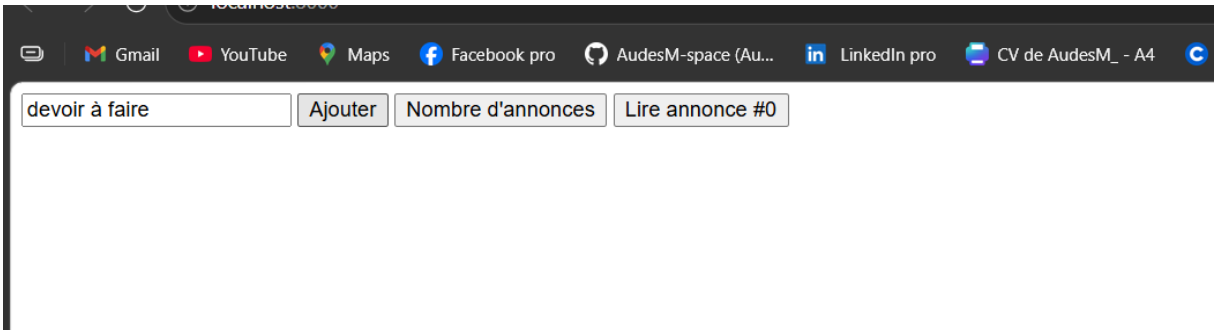


Figure 22: Déploiement & test via Remix

7. Création d'une interface Web minimale

Enfin, une page HTML utilisant Web3.js est développée pour interagir avec le contrat via MetaMask. L'utilisateur peut saisir un message pour publier une annonce, consulter une annonce par identifiant et afficher le nombre total d'annonces. L'intérêt est de montrer la concrétisation du projet : une application décentralisée utilisable par les étudiants pour partager et consulter des informations de manière transparente et sécurisée.



La réalisation de cette DApp a permis de franchir une étape importante : passer d'un contrat simple à une application décentralisée complète, intégrant un cas d'usage concret. Nous avons appris à structurer un contrat plus riche, à gérer des données complexes (identifiant, message, auteur, date), à automatiser son déploiement et à valider son fonctionnement par des tests

unitaires. L'intégration avec MetaMask et la création d'une interface web minimale ont montré comment une DApp peut être utilisée par des étudiants pour publier et consulter des annonces en toute confiance. Cette partie met en évidence la valeur ajoutée de la blockchain dans la gestion de l'information : transparence, traçabilité et absence d'intermédiaire. Elle constitue une base solide pour envisager des projets plus ambitieux, comme des plateformes collaboratives ou des systèmes de gestion académique décentralisés.

CONCLUSION

Ce travail pratique nous a permis de parcourir l'ensemble du cycle de développement d'une application décentralisée, depuis la mise en place de l'environnement technique jusqu'à la création d'une interface web fonctionnelle. La première partie, centrée sur un contrat simple, a posé les bases indispensables : installation des outils, compilation, migration et tests. Elle a servi de tremplin pour comprendre les mécanismes fondamentaux de la blockchain et des smart contracts.

La deuxième partie a marqué une étape plus ambitieuse, avec la conception d'une DApp de gestion des annonces de la classe. Ce mini-projet a montré comment un besoin concret peut être traduit en logique blockchain, et comment une application décentralisée peut offrir transparence, traçabilité et fiabilité dans la gestion de l'information. L'intégration avec MetaMask et la mise en place d'une interface web minimale ont permis de simuler une expérience utilisateur réelle, confirmant la pertinence de la démarche.

Au-delà de l'aspect technique, ce projet illustre l'intérêt pédagogique de la blockchain : apprendre à manipuler une technologie innovante tout en l'appliquant à un cas d'usage proche du quotidien académique. Il ouvre également des perspectives pratiques : la même logique pourrait être étendue à des plateformes collaboratives, des systèmes de gestion académique décentralisés ou encore des outils de communication sécurisés pour des communautés plus larges.

En somme, ce mini-projet démontre que la blockchain n'est pas seulement une technologie abstraite, mais un outil concret qui peut transformer la manière dont nous gérons et partageons l'information. Les compétences acquises ici constituent une base solide pour explorer des projets plus complexes et contribuer à l'adoption de solutions décentralisées dans l'éducation et au-delà.